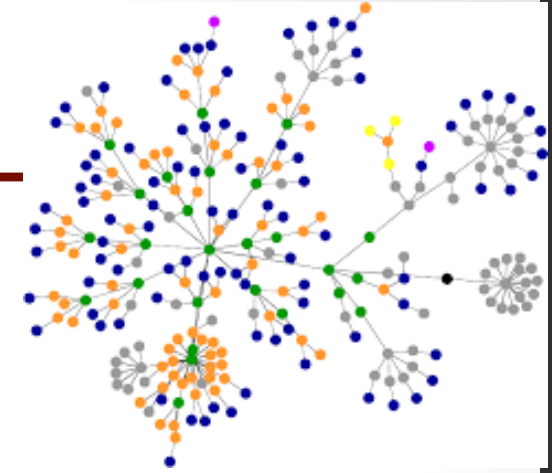


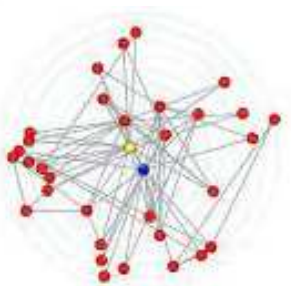


ESTRUTURA DE DADOS

ENGENHARIA DA COMPUTAÇÃO

Teoria dos Grafos – Algoritmos de Busca e do Menor Caminho (Dijkstra)

Profa. Dra. Lúcia F. A. Guimarães





- **Algoritmos de Busca**

- **Objetivo:**

- Explorar um grafo, isto é, obter um processo sistemático de como caminhar por seus vértices e arestas
- Pesquisa sistemática de cada aresta e vértice de um grafo.
- Válido tanto para dígrafo quanto para grafo não direcionado





- **Algoritmos de Busca**

- Existem basicamente dois tipos de busca

- **Largura**

- Busca primeiro nos vértices localizados a uma mesma distância da raiz de busca

- **Profundidade**

- Busca o mais profundo no grafo sempre que possível





- **Busca em Largura**

- Seja um grafo $G = (V, E)$ e um vértice origem v .
- Pesquisa em largura:
 - É realizada visitando (isto é, avaliando os dados associados com) os vértices mais próximos daquele que a busca foi inicializada antes de visitar qualquer outro.
 - A ideia então é aplicada recursivamente





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”
ESCOLHA UM VÉRTICE V EM V
VISITE V
MARQUE V COMO “VISITADO”
COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ **FRENTE** (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

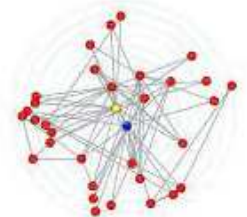
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ **FRENTE** (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

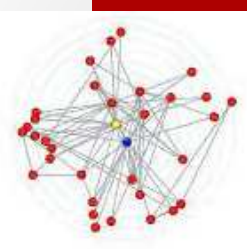
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ **FRENTE** (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

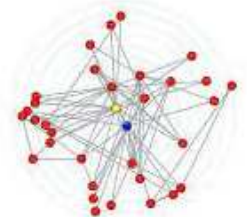
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ **FRENTE** (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

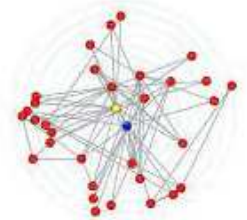
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ **FRENTE** (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

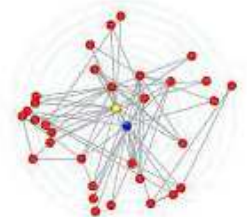
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ **FRENTE** (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

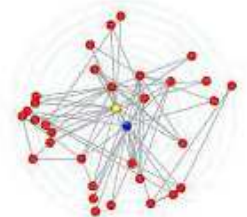
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

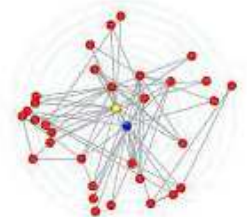
COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

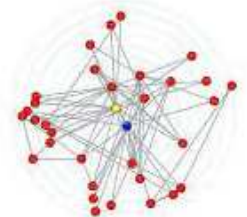
COMO “VISITADO”

INSIRA W NO **FIM** DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

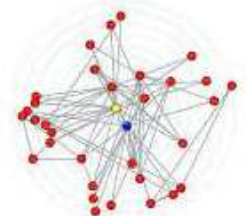
COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

FIM DO REPITA





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

MARQUE TODOS OS VÉRTICES DE
 V COMO “NÃO VISITADOS”

ESCOLHA UM VÉRTICE V EM V

VISITE V

MARQUE V COMO “VISITADO”

COLOQUE V EM Q

REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

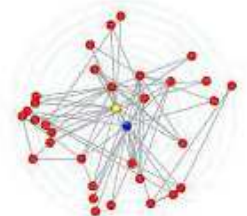
COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

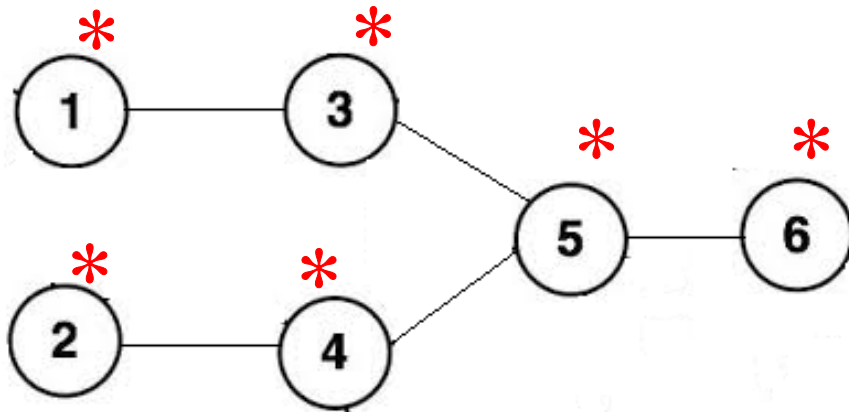
APAGUE U DE Q

FIM DO REPITA





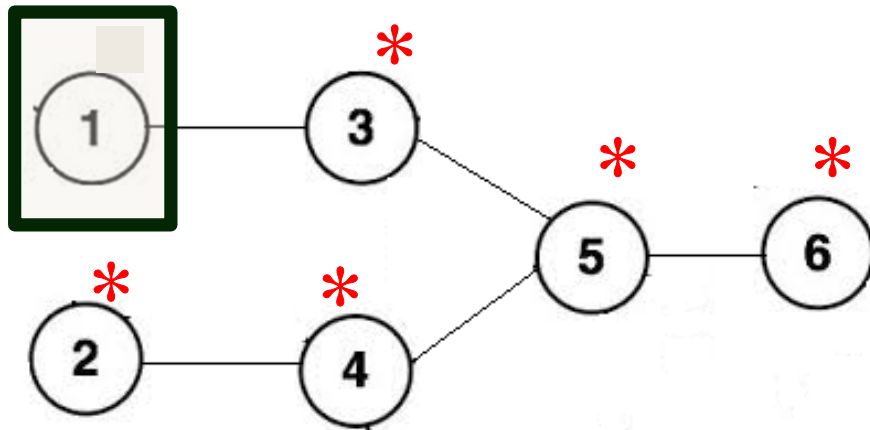
- Busca em Largura – Exemplo**



MARQUE TODOS OS VÉRTICES DE V COMO “NÃO VISITADOS”



• Busca em Largura – Exemplo



ESCOLHA UM VÉRTICE V EM V

VISITE V

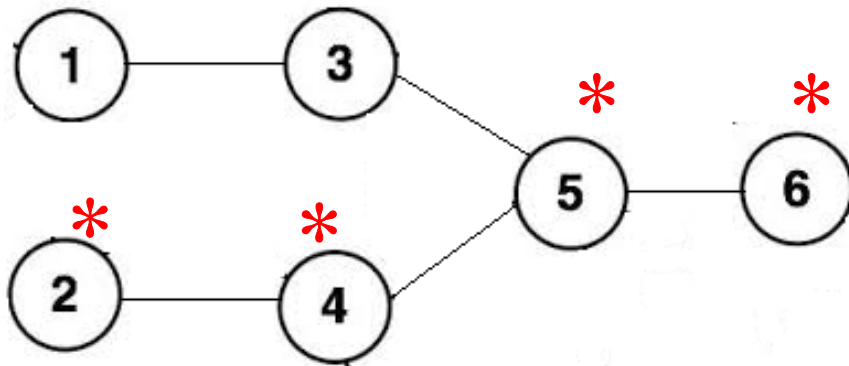
MARQUE V COMO “VISITADO”

COLOQUE V EM Q

Q: 1



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

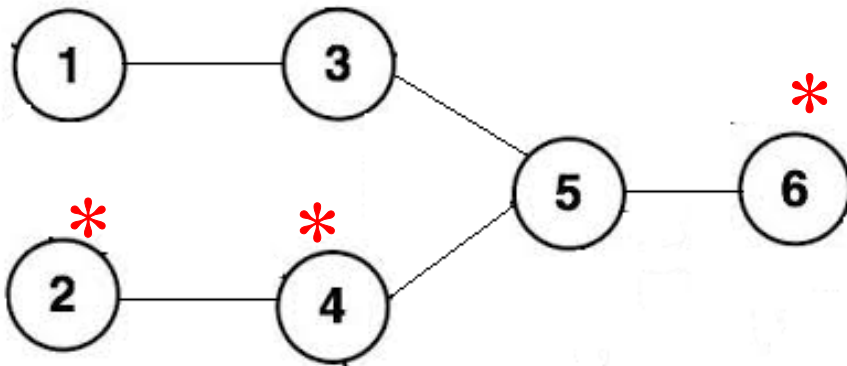
FIM DO REPITA

$u \leftarrow 1$ $A(u) = \{3\}$

Q : ~~X~~, 3



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $w \in A(U)$

SE w NÃO FOI VISITADO ENTÃO

VISITE w ;

MARQUE w

COMO “VISITADO”

INSIRA w NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

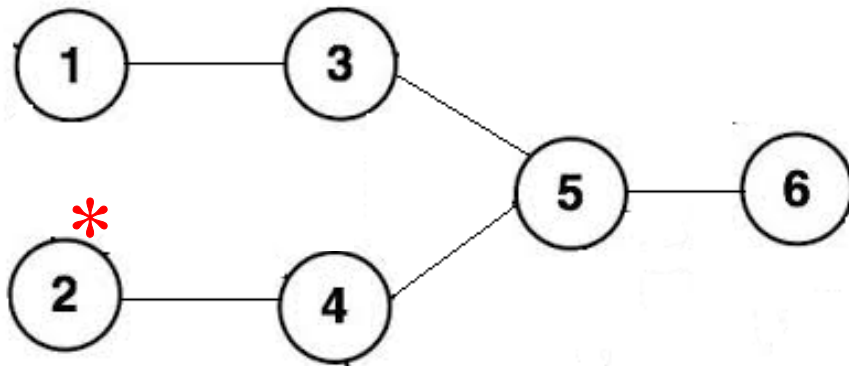
FIM DO REPITA

$u \leftarrow 3$ $A(u) = \{1, 5\}$ $w = 1$ $w = 5$

Q : ~~3~~, 5



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $w \in A(U)$

SE w NÃO FOI VISITADO ENTÃO

VISITE w ;

MARQUE w

COMO “VISITADO”

INSIRA w NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

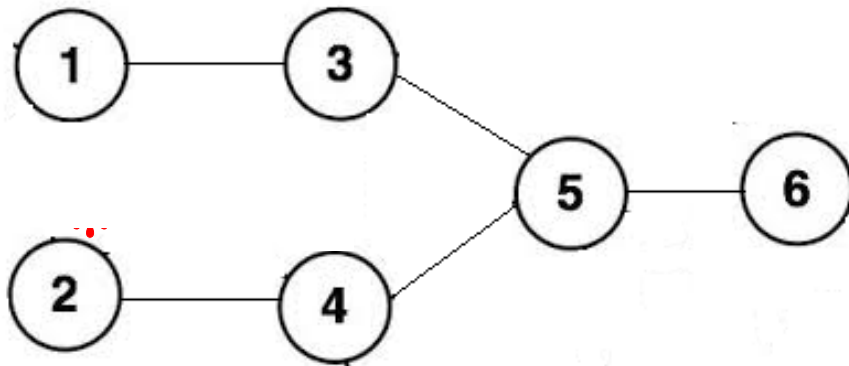
FIM DO REPITA

$u \leftarrow 5$ $A(u) = \{3, 4, 6\}$ $w = 3$ $w = 4$ $w = 6$

Q : ~~2~~, 4, 6



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $w \in A(U)$

SE w NÃO FOI VISITADO ENTÃO

VISITE w ;

MARQUE w

COMO “VISITADO”

INSIRA w NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

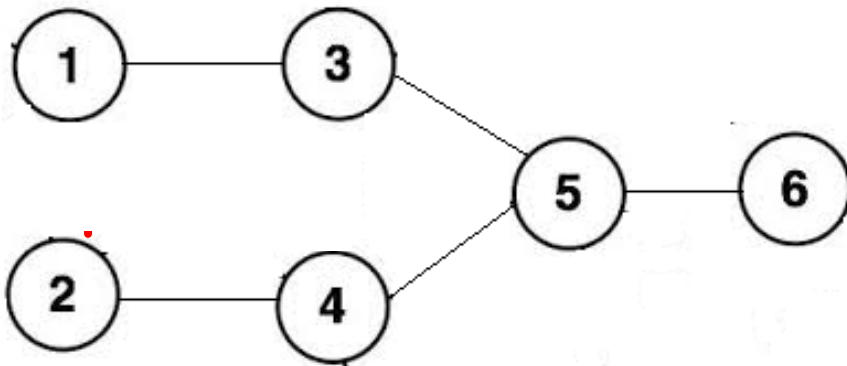
FIM DO REPITA

$u \leftarrow 4$ $A(u) = \{2, 5\}$ $w = 2$ $w = 5$

Q : ~~X~~, 6, 2



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

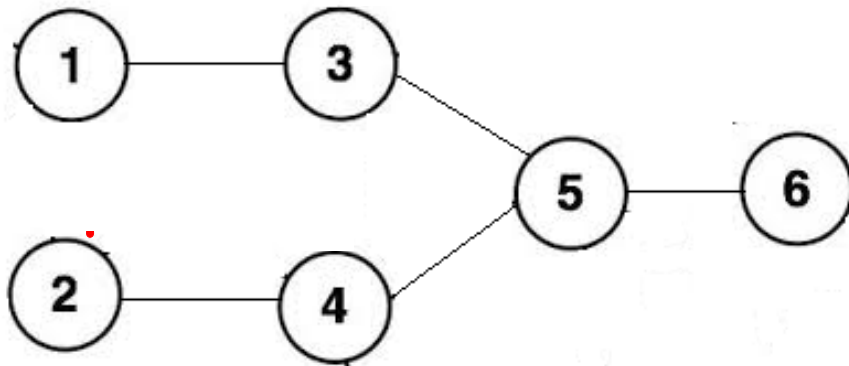
FIM DO REPITA

$u \leftarrow 6$ $A(u) = \{ 5 \}$

$Q: \text{X}, 2$



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

APAGUE U DE Q

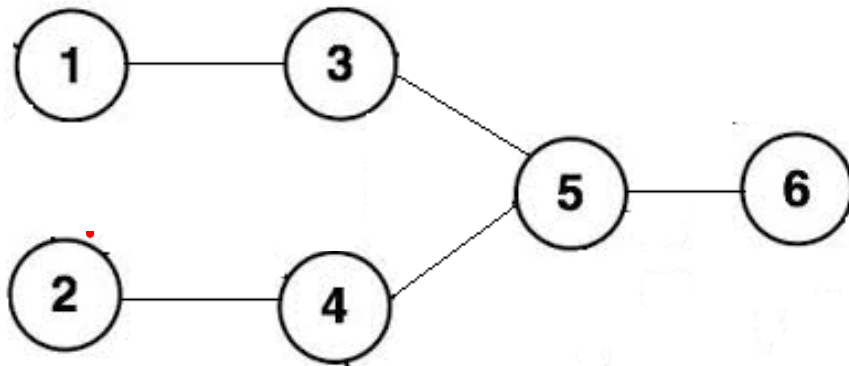
FIM DO REPITA

$u \leftarrow 2$ $A(u) = \{ 4 \}$

Q : ~~X~~



• Busca em Largura – Exemplo



REPITA ENQUANTO $Q \neq \emptyset$

$U \leftarrow$ FRENTE (INÍCIO) Q

PARA $W \in A(U)$

SE W NÃO FOI VISITADO ENTÃO

VISITE W ;

MARQUE W

COMO “VISITADO”

INSIRA W NO FIM DE Q

FIM DO PARA

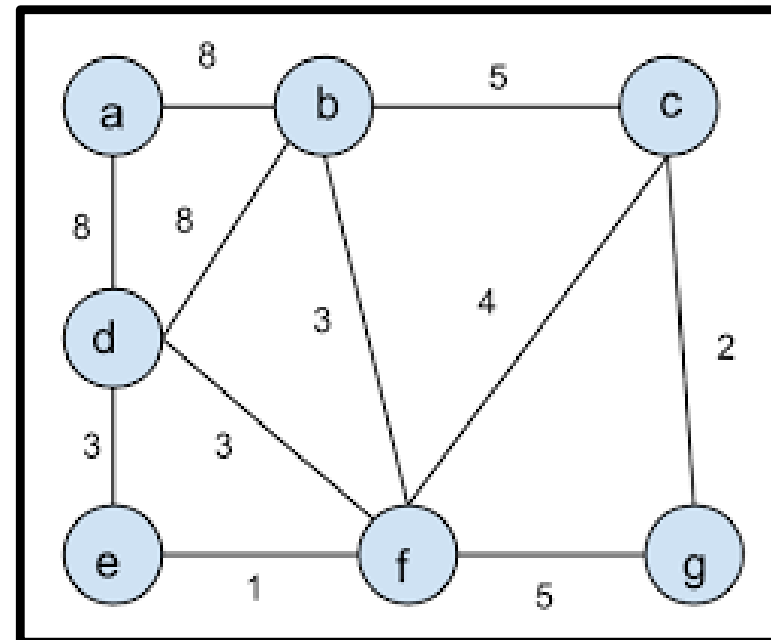
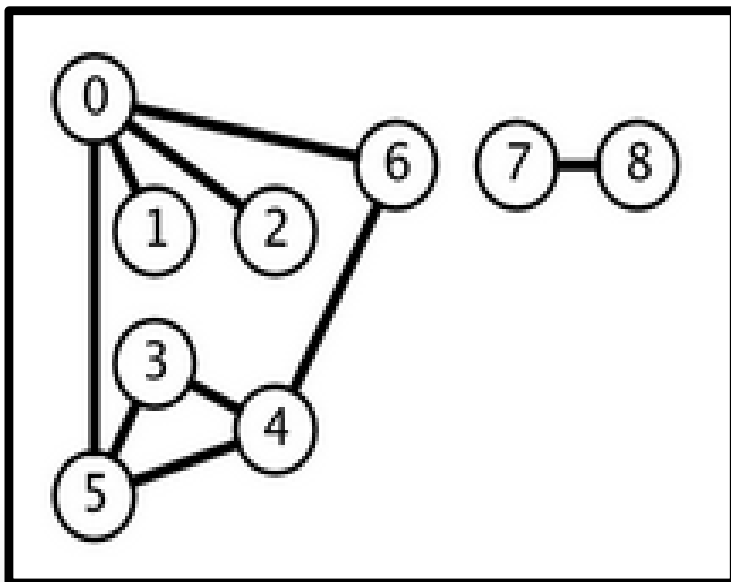
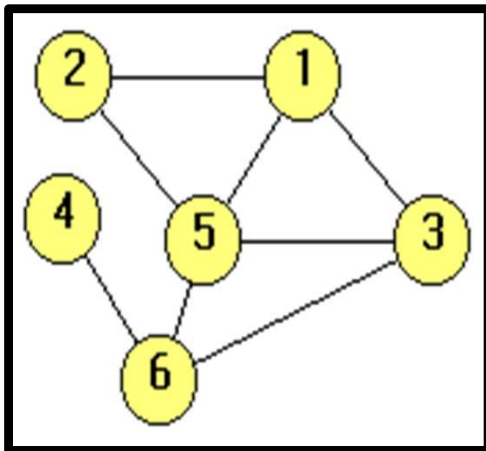
APAGUE U DE Q

FIM DO REPITA

FILA VAZIA E TODOS OS NÓS VISITADOS

Q:

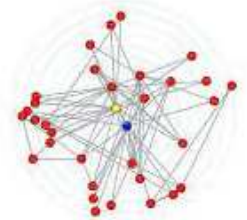
- **Exercício – Faça Você Agora o Teste de mesa**
inicie do menor vértice





- **Busca em Profundidade**

- Seja um grafo $G = (V, E)$ e um vértice origem v .
- **Pesquisa em Profundidade:**
 - Iniciar a busca em v e ir tão longe quanto puder até que não haja vértices adjacentes para serem visitados.
 - A partir daí, então, voltar no caminho até encontrar um vértice vizinho que não foi visitado e novamente aprofundar o caminho até onde der
 - **Ideia recursiva** e a melhor forma de implementação.
 - A **principal característica** deste algoritmo é que **nunca uma direção é pesquisada mais de uma vez e TODAS as possíveis soluções são pesquisadas.**





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO

COMO

NÃO VISITADO ENTÃO

PROF(w)

FIM DO PARA

RETORNE

DO PROCEDIMENTO

INICIO

MARQUE TODOS OS VÉRTICES DE V

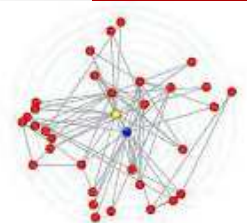
COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM

VÉRTICE v

PROF(v)

FIM





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO

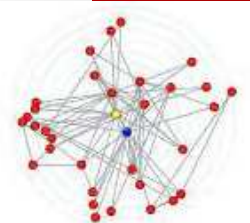
INICIO

MARQUE TODOS OS VÉRTICES DE V
COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

PROF(v)

FIM





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO

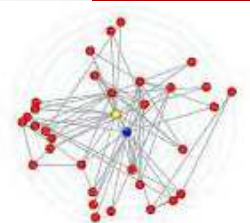
INICIO

MARQUE TODOS OS VÉRTICES DE V
COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

PROF(v)

FIM





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



INICIO

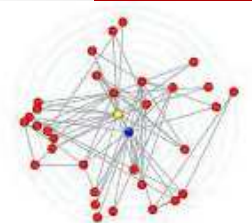
MARQUE TODOS OS VÉRTICES DE V

COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

PROF(v)

FIM





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



INICIO

MARQUE TODOS OS VÉRTICES DE V

COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

PROF(v)

FIM





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



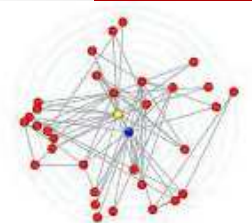
INICIO

MARQUE TODOS OS VÉRTICES DE V
COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

PROF(v)

FIM





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO $\text{PROF}(u)$

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
 $\text{PROF}(w)$

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



INICIO

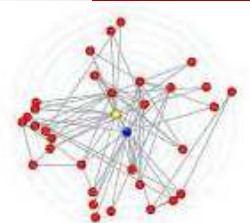
MARQUE TODOS OS VÉRTICES DE V

COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

$\text{PROF}(v)$

FIM





Grafos – Algoritmos de Busca

• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



INICIO

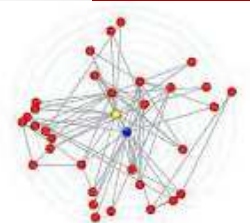
MARQUE TODOS OS VÉRTICES DE V

COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

PROF(v)

FIM





• Algoritmo

ENTRADA: $G(V,E)$

SAÍDA: O RESULTADO DOS DADOS
AVALIADOS EM CADA VÉRTICE

PROCEDIMENTO PROF(u)

VISITE u

MARQUE u COMO “VISITADO”

PARA $w \in A(u)$

SE w ESTÁ MARCADO
COMO

NÃO VISITADO ENTÃO
PROF(w)

FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



INICIO

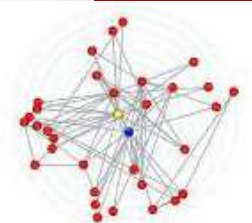
MARQUE TODOS OS VÉRTICES DE V

COMO NÃO VISITADOS

ESCOLHA, ARBITRARIAMENTE, UM
VÉRTICE v

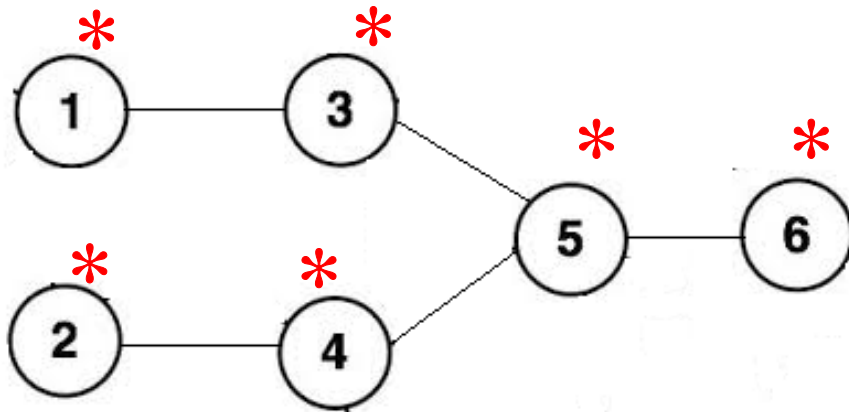
PROF(v)

FIM





• Busca em Profundidade – Exemplo

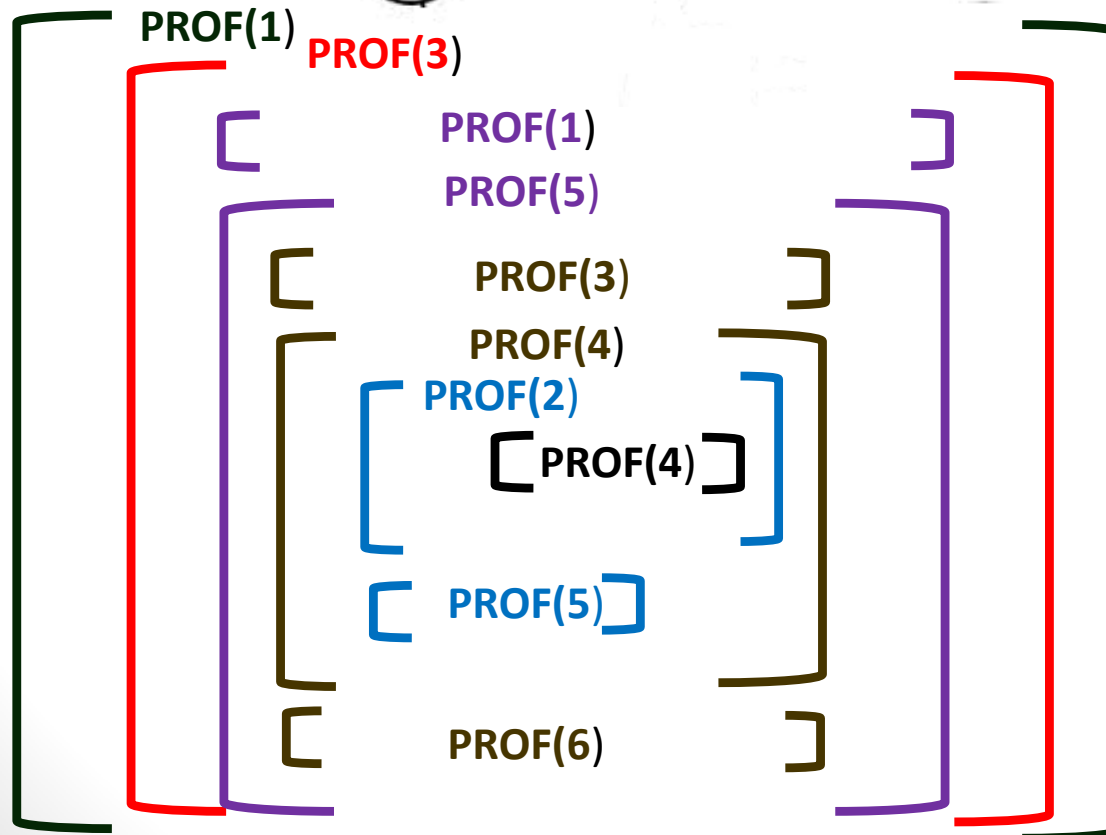
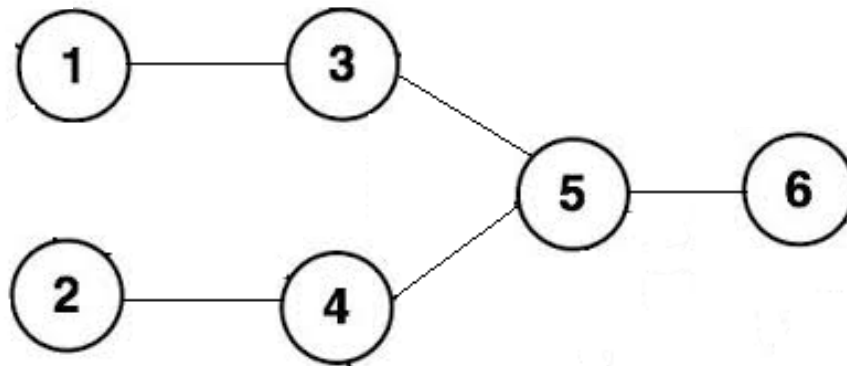


MARQUE TODOS OS VÉRTICES DE V COMO “NÃO VISITADOS”

ESCOLHA, ARBITRARIAMENTE, UM VÉRTICE v
 $PROF(v)$



• Busca em Profundidade – Exemplo



PROCEDIMENTO PROF(U)

VISITE U

MARQUE U COMO “VISITADO”

PARA $W \in A(U)$

SE W ESTÁ MARCADO

COMO NÃO VISITADO

ENTÃO PROF(W)

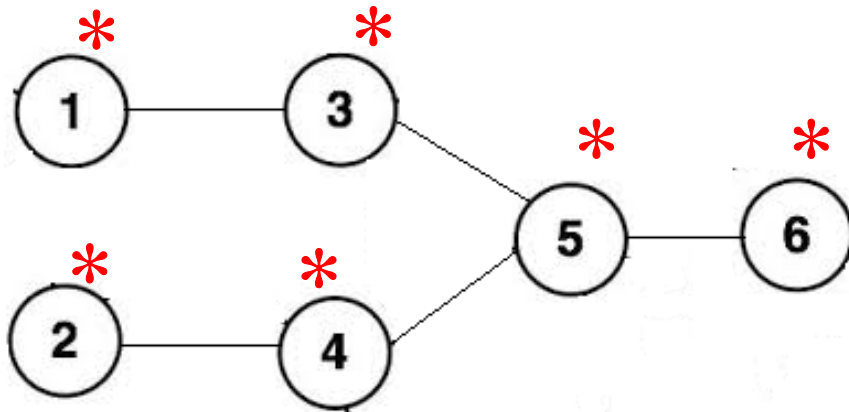
FIM DO PARA

RETORNE

FIM DO PROCEDIMENTO



• Busca em Profundidade – Exemplo

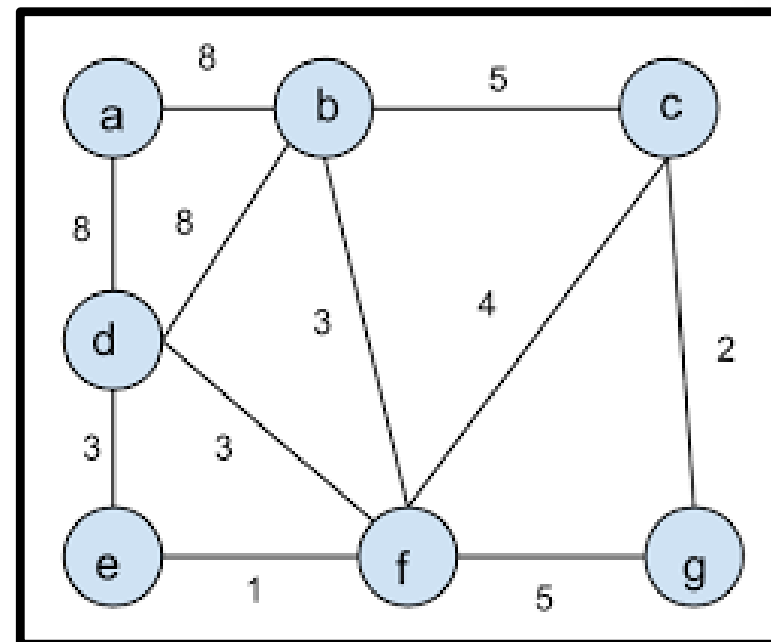
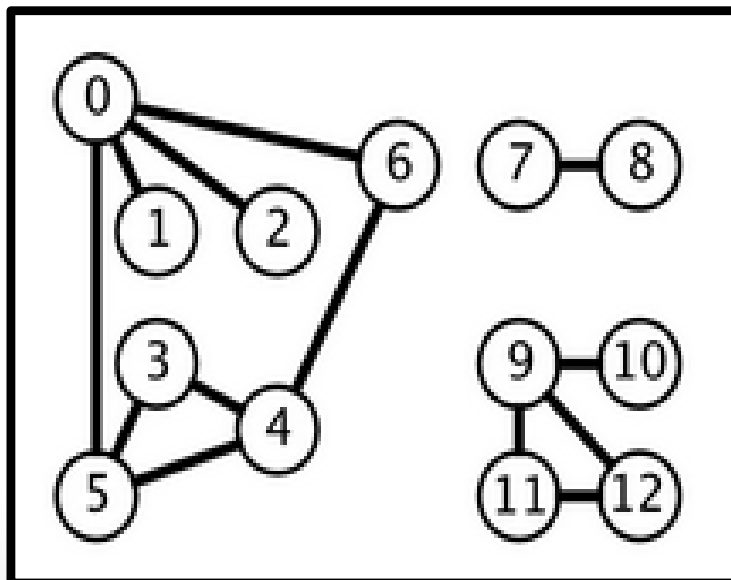
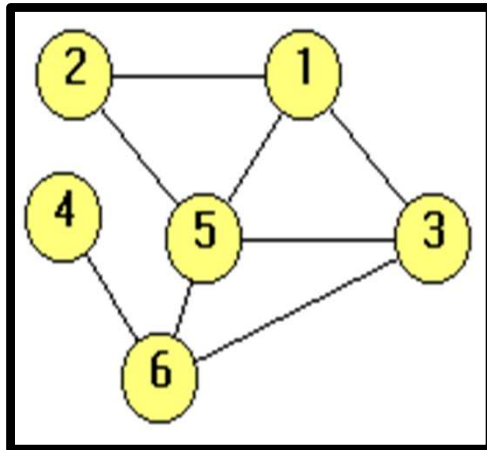


MARQUE TODOS OS VÉRTICES DE V COMO “NÃO VISITADOS”

ESCOLHA, ARBITRARIAMENTE, UM VÉRTICE v
 $PROF(v)$



- **Exercício – Faça Você Agora o Teste de mesa**
inicie do menor vértice





- **DIJKSTRA**

- Calcula o caminho de custo mínimo **entre dois vértices de um grafo.**
- É bastante simples e com um bom nível de performance.
- Ele parte de uma estimativa para o custo mínimo e vai sucessivamente ajustando esta estimativa.
- **Um vértice é considerado fechado quando já tiver sido obtido o seu caminho de custo mínimo**, a partir do vértice origem.

Caso contrário ele é considerado **aberto**





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

- **Objetivo:**

- Encontrar o valor do menor caminho entre 2 vértices onde existe um “peso”, custo, associado a cada arco.
- A **exatidão da solução é garantida**, somente quando “pesos”, não negativos, são considerados.
- O algoritmo foi construído considerando **dígrafos**.



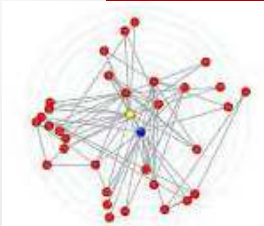


Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

- **Idéia Básica:**

- Dado um vértice origem **V_o** , **encontrar o menor caminho entre V_o e os outros vértices, até que o vértice obtido seja o vértice destino (V_d).**

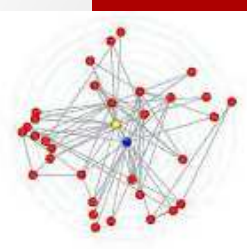




- **DIJKSTRA**

- **Definições**

- U – conjunto de vértices mais próximos ao vértice origem.
 - os vértices pertencentes à U são **fechados**;
- DV_i – distância do menor caminho entre o **V_o** e V_i
- **u** – variável que representa o vértice atual.
- $TAM(u, V_i)$ – “tamanho”, “peso” do arco que liga u à **V_i** .
- Cálculo de $DV_i = \min\{ DV_{i_j}, D_u + TAM(u, V_i) \}$, que significa o mínimo entre DV_i e $(D_u + TAM(u, V_i))$,





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

ENTRADA: $G(V,E)$, V_o , V_D

$D_{V_o} \leftarrow 0$

Para $i = 1$ até V faça

se $V_i \neq V_o$ então $D_{V_i} \leftarrow \infty$

Fim Para

$U \leftarrow V_o$ {Conjunto U , vértices fechados recebe o vértice
origem}

$u \leftarrow V_o$ {vértice atual, de pesquisa, recebe o vértice
origem}





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

ENTRADA: $G(V,E)$, V_o , V_D

$D_{V_o} \leftarrow 0$

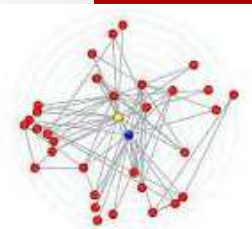
Para $i = 1$ até V faça

Se $V_i \neq V_o$ então $D_{V_i} \leftarrow \infty$

Fim Para

$U \leftarrow V_o$ {Conjunto U , vértices fechados recebe o vértice
origem}

$u \leftarrow V_o$ {vértice atual, de pesquisa, recebe o vértice
origem}





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

ENTRADA: $G(V,E)$, V_O , V_D

$DV_O \leftarrow 0$

Para $i = 1$ até V faça

Se $V_i \neq V_O$ então $DV_i \leftarrow \infty$

Fim Para

$U \leftarrow V_O$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_O$ {vértice atual, de pesquisa, recebe o vértice origem}





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

ENTRADA: $G(V,E)$, V_O , V_D

$D_{V_O} \leftarrow 0$

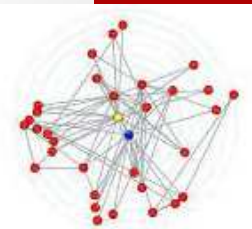
Para $i = 1$ até V faça

Se $V_i \neq V_O$ então $D_{V_i} \leftarrow \infty$

Fim Para

$U \leftarrow V_O$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_O$ {vértice atual, de pesquisa, recebe o vértice origem}





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

ENTRADA: $G(V,E)$, V_O , V_D

$DV_O \leftarrow 0$

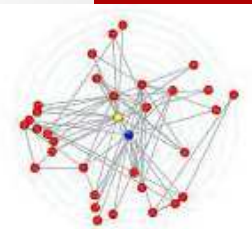
Para $i = 1$ até V faça

Se $V_i \neq V_O$ então $DV_i \leftarrow \infty$

Fim Para

$U \leftarrow V_O$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_O$ {vértice atual, de pesquisa, recebe o vértice origem}





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então
 $Menor_d \leftarrow DV_i$
 $u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então
 $Menor_d \leftarrow DV_i$
 $u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$



Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i

Fim Para

$\text{Menor_d} \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < \text{Menor_d})$ então
 $\text{Menor_d} \leftarrow DV_i$
 $u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$



Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

**Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i**

Fim Para

$\text{Menor_d} \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < \text{Menor_d})$ então

$\text{Menor_d} \leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

calcule DV_i

Fim Para

$Menor_d \leftarrow \infty$

$$DV_i = \min\{ DV_i, Du + TAM(u, V_i) \}$$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

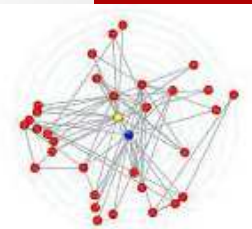
$Menor_d \leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{ u \}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < \text{Menor_d})$ então
Menor_d $\leftarrow DV_i$
 $u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule D_{Vi}

Fim Para

$\text{Menor_d} \leftarrow \infty$

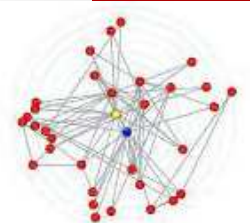
Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(D_{Vi} < \text{Menor_d})$ então
 $\text{Menor_d} \leftarrow D_{Vi}$
 $U \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule D_{Vi}

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(D_{Vi} < Menor_d)$ então

$Menor_d \leftarrow D_{Vi}$

$U \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então
 $Menor_d \leftarrow DV_i$
 $u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então
calcule DV_i

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

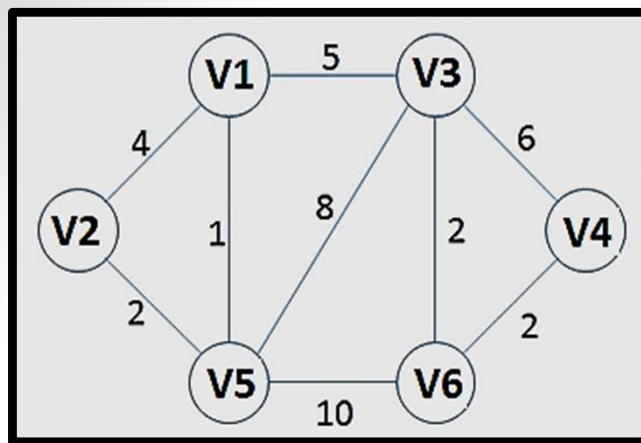
Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então
 $Menor_d \leftarrow DV_i$
 $u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo

$V_o = V2$

$V_d = V4$

ENTRADA: $G(V,E)$, V_o , V_d

$DV_o \leftarrow 0$

Para $i = 1$ até V faça

se $V_i \neq V_o$ então $DV_i \leftarrow \infty$

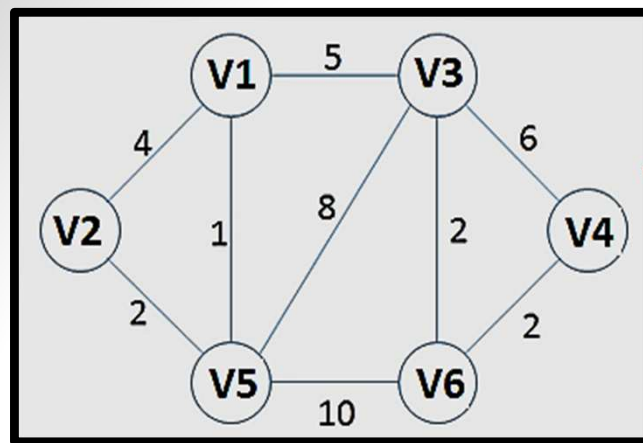
Fim Para

$U \leftarrow V_o$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_o$ {vértice atual, de pesquisa, recebe o vértice origem}

DV1	DV2	DV3	DV4	DV5	DV6
	0				

**Cada Linha da Tabela
será uma iteração do
laço (Repita)**



Algoritmo

$V_o = V2$

$V_d = V4$

ENTRADA: $G(V,E)$, V_o , V_d

$DV_o \leftarrow 0$

Para $i = 1$ até V faça

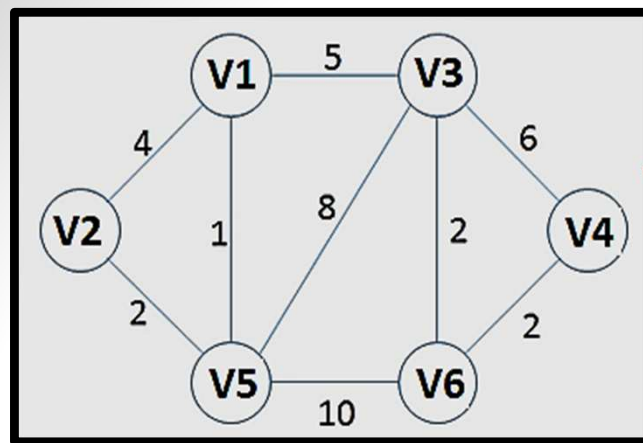
se $V_i \neq V_o$ então $DV_i \leftarrow \infty$

Fim Para

$U \leftarrow V_o$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_o$ {vértice atual, de pesquisa, recebe o vértice origem}

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞



Algoritmo

$V_o = V2$

$V_d = V4$

ENTRADA: $G(V,E)$, V_o , V_d

$DV_o \leftarrow 0$

Para $i = 1$ até V faça

se $V_i \neq V_o$ então $DV_i \leftarrow \infty$

Fim Para

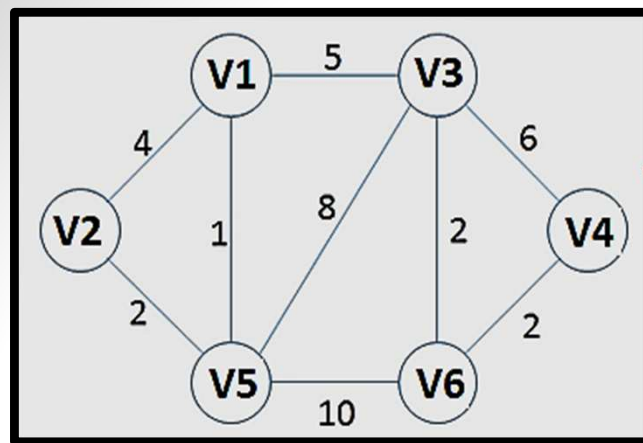
$U \leftarrow V_o$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_o$ {vértice atual, de pesquisa, recebe o vértice origem}

$U = \{V2\}$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞

$u = V2$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

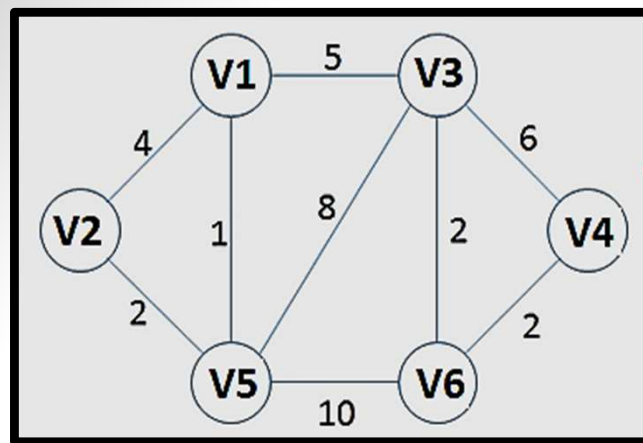
Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞

$u = V2$

$DV1 = \min\{\infty, 0+4\} = 4$

$DV5 = \min\{\infty, 0+2\} = 2$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞

$u = V2$

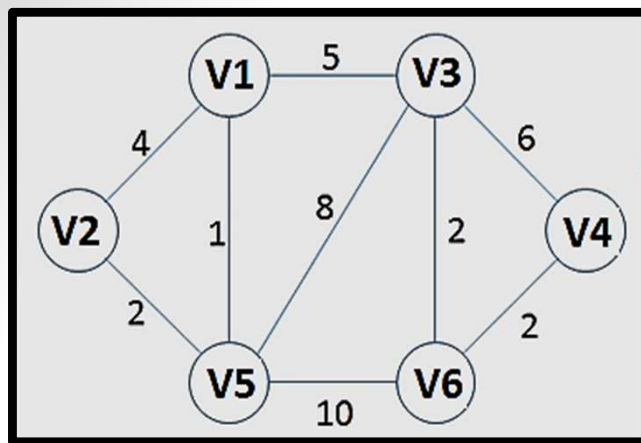
$DV1 = \text{Min}\{\infty, 0+4\} = 4$

$DV5 = \text{Min}\{\infty, 0+2\} = 2$

Menor_d = ∞

Menor_d = 4 $u = V1$

Menor_d = 2 $u = V5$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12

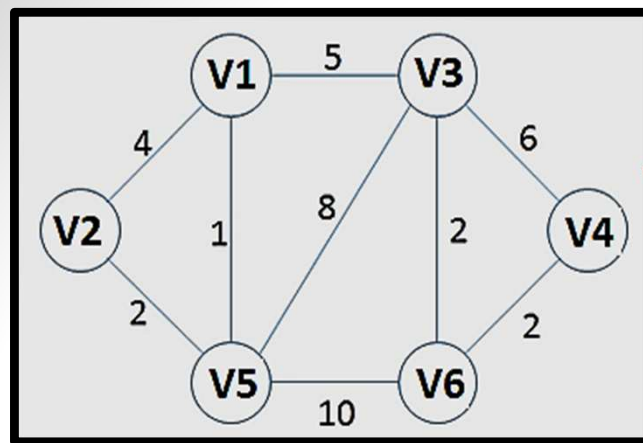
$u = V2$

$u = V5$

$DV1 = \min\{4, 2+1\} = 3$

$DV3 = \min\{\infty, 2+8\} = 10$

$DV6 = \min\{\infty, 2+10\} = 12$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

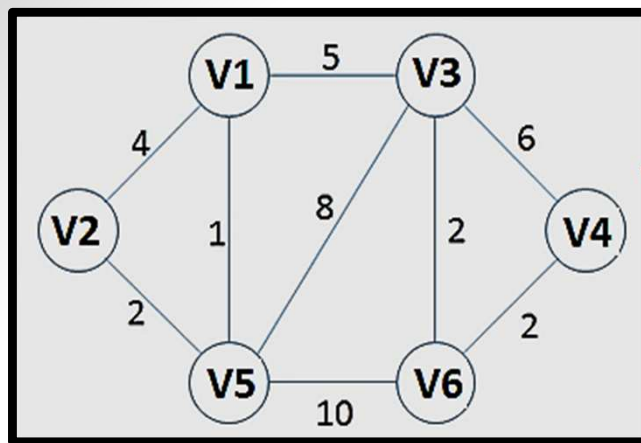
DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12

$u = V2$

$u = V5$

Menor_d = ∞

Menor_d = 3 $u = V1$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

$Menor_d \leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

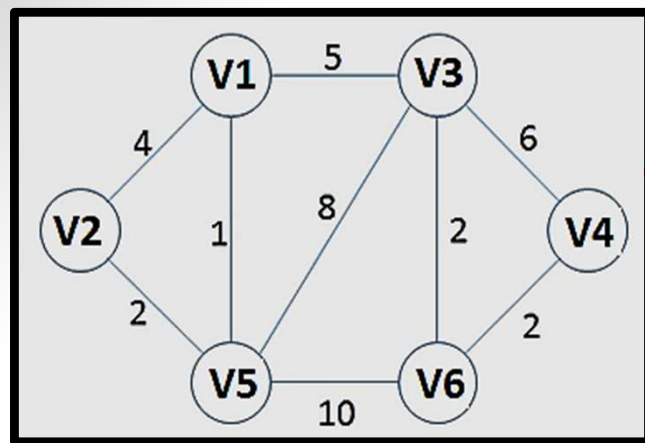
DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12
3	0	8	∞	2	12

$u = V2$

$u = V5$

$u = V1$

$DV3 = \min\{10, 3+5\} = 8$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1, V3\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12
3	0	8	∞	2	12

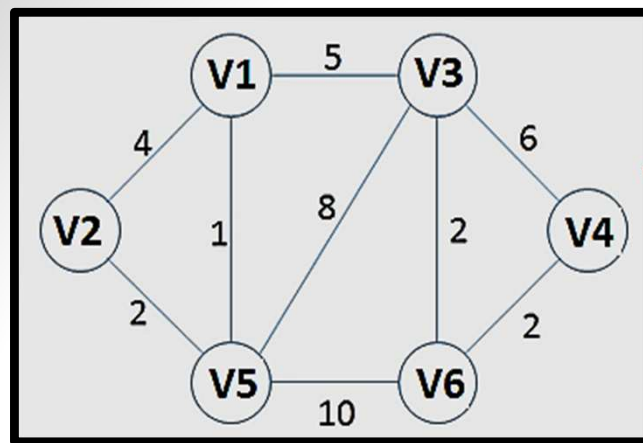
$u = V2$

$u = V5$

$u = V1$

Menor_d = ∞

Menor_d = 8 $u = V3$



Algoritmo

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

$U = \{V2, V5, V1, V3\}$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12
3	0	8	∞	2	12
3	0	8	14	2	10

$u = V2$

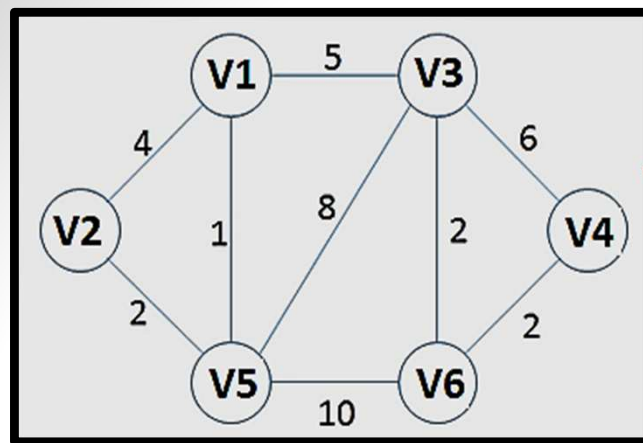
$u = V5$

$u = V1$

$u = V3$

$DV4 = \min\{\infty, 8+6\} = 14$

$DV6 = \min\{12, 8+2\} = 10$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1, V3, V6\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{ DV_i, Du + TAM(u, V_i) \}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12
3	0	8	∞	2	12
3	0	8	14	2	10

$u = V2$

$u = V5$

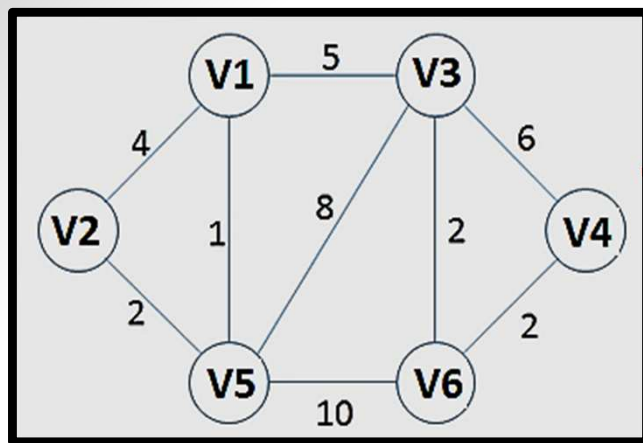
$u = V1$

$u = V3$

Menor_d = ∞

Menor_d = 14 $u = V4$

Menor_d = 10 $u = V6$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1, V3, V6\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12
3	0	8	∞	2	12
3	0	8	14	2	10
3	0	8	12	2	10

$u = V2$

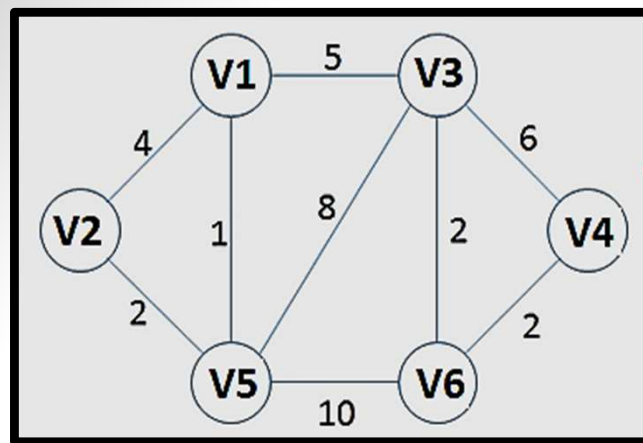
$u = V5$

$u = V1$

$u = V3$

$u = V6$

$DV4 = \min\{14, 10 + 2\} = 12$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1, V3, V6, V4\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
∞	0	∞	∞	∞	∞
4	0	∞	∞	2	∞
3	0	10	∞	2	12
3	0	8	∞	2	12
3	0	8	14	2	10
3	0	8	12	2	10

$u = V2$

$u = V5$

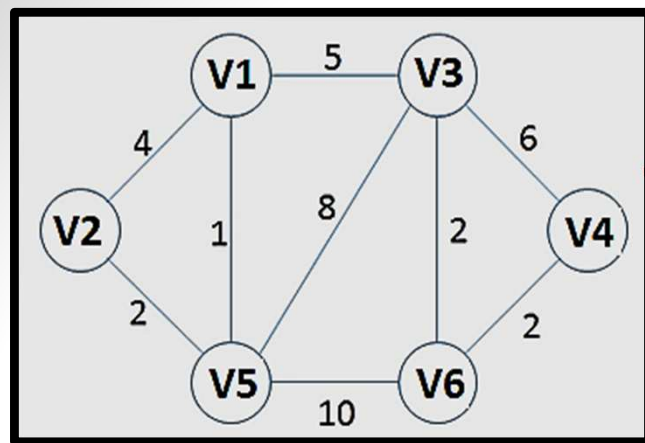
$u = V1$

$u = V3$

$u = V6$

Menor_d = ∞

Menor_d = 14 $u = V4$



Algoritmo

$V_o = V2$

$V_d = V4$

$U = \{V2, V5, V1, V3, V6, V4\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ então

$\min\{DV_i, Du + TAM(u, V_i)\}$

Fim Para

Menor_d $\leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

Menor_d $\leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

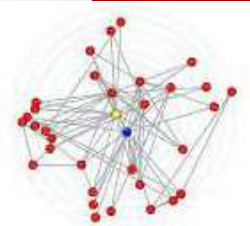
Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
3	0	8	12	2	10



- **DIJKSTRA MODIFICADO**

- Este algoritmo pode ser modificado, para **encontrar todos os caminhos mínimos de um grafo a partir de certo vértice origem.**
- Para isso basta
 - Marcar todos os vértices como aberto, com exceção do vértice origem que já entra no laço principal como fechado.
 - E mudar a condição do repita para até que não exista vértice aberto

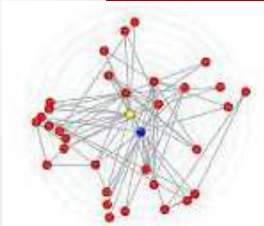




Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA MODIFICADO**

- DIJKSTRA não fornece **qual** o menor caminho, apenas o “pes/custo” do mesmo.
- Uma modificação, neste algoritmo, pode ser feita para que ele imprima o menor caminho.
- Um novo vetor é inserido **(CAM)** e este vetor guardará os vértices usados para atingir o menor caminho





Algoritmo do Menor Caminho — DIJKSTRA

- **DIJKSTRA**

ENTRADA: $G(V,E)$, V_o , V_D

$D_{V_o} \leftarrow 0$

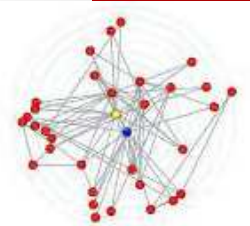
Para $i = 1$ até V faça

Se $V_i \neq V_o$ então $D_{V_i} \leftarrow \infty$

$CAM[i] \leftarrow -1$ // caminho de V_i

$U \leftarrow V_o$ {Conjunto U , vértices fechados recebe o vértice origem}

$u \leftarrow V_o$ {vértice atual, de pesquisa, recebe o vértice origem}





Algoritmo do Menor Caminho — DIJKSTRA

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ e

$(DV_i > Du + TAM(u, V_i))$ então

$DV_i \leftarrow Du + TAM(u, V_i)$

$CAM[V_i] \leftarrow u$

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

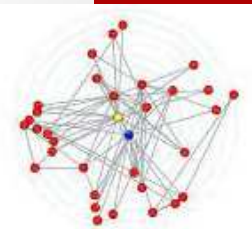
$Menor_d \leftarrow DV_i$

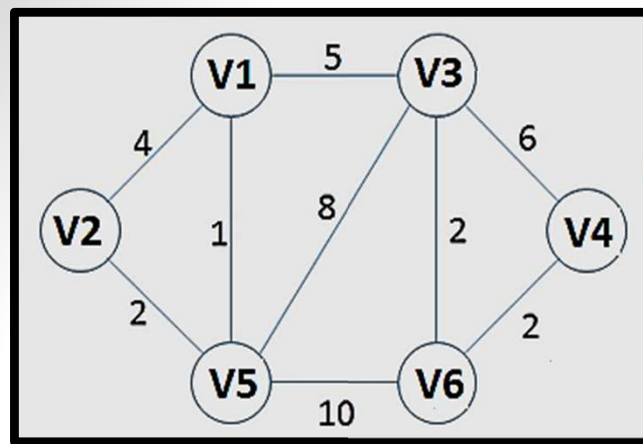
$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$





Algoritmo

$U = \{V2, V5, V1, V3, V6, V4\}$

Repita

Para $i = 1$ até V faça

Se $(V_i \neq u)$ e $(u$ é adjacente a $V_i)$ e $(V_i \notin U)$ e

$(DV_i > Du + TAM(u, V_i))$ então

$DV_i \leftarrow Du + TAM(u, V_i)$

$CAM[V_i] \leftarrow u$

Fim Para

$Menor_d \leftarrow \infty$

Para $i = 1$ até V faça

Se $(V_i \notin U)$ e $(DV_i < Menor_d)$ então

$Menor_d \leftarrow DV_i$

$u \leftarrow V_i$

Fim Para

$U \leftarrow U \cup \{u\}$

Até que $u = V_d$

DV1	DV2	DV3	DV4	DV5	DV6
3	0	8	12	2	10

CAM V1	CAM DV2	CAM DV3	CAM DV4	CAM DV5	CAM DV6
5	-1	1	6	2	3

v2,v,5,v1,v3,v6,v4 com custo 12



Algoritmo do Menor Caminho — DIJKSTRA



• Exercícios

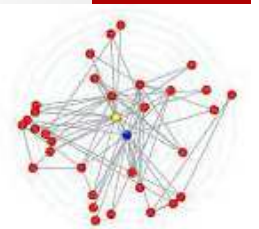
1. O algoritmo do menor caminho foi executado para um grafo de 10 vértices O resultado obtido foi

dv1	dv2	dv3	dv4	dv5	dv6	dv7	dv8	dv9	dv10
3	0	∞	3	5	4	∞	20	8	7

Cam	Cam	Cam	Cam	Cam	Cam	Cam	Cam	Cam	Cam
1	2	3	4	5	6	7	8	9	10
2	-1	-1	6	6	1	-1	5	10	6

$U = \{v_2, v_1, v_4, v_6, v_5, v_{10}, v_9\}$.

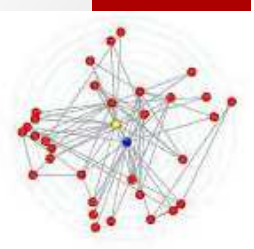
Responda e justifique **Atenção todas as respostas deverão ter justificativas caso contrário não serão aceitas**





• Exercícios

- a) O que significa $dv6 = 4$?
- b) Qual o vértice origem?
- c) Qual o vértice destino?
- d) Qual **o menor caminho** entre o vértice origem e o vértice destino e **qual seu valor**.
- e) O que você pode concluir em relação à v7?
- f) O que você pode concluir em relação à v8?





• Exercícios

2. Considere a matriz na qual (i,j) é o peso do arco direcionado de i para j . Se $(i,j) = \text{"-"}$ então não existe arco de i para j . Para a matriz adjacente modificada abaixo, determine o tamanho (peso/valor) do menor caminho de v_0 à v_4 e qual o menor caminho, **sem desenhar qualquer parte do grafo.**

$$\begin{bmatrix} 1 & 3 & - & 2 & - \\ 3 & - & 2 & 1 & 3 \\ 1 & - & 3 & 2 & 5 \\ 5 & 1 & - & 5 & 6 \\ 6 & - & 3 & 2 & - \end{bmatrix}$$

